

Homework 4 — Unsupervised Learning

Statistical and Mathematical Methods for AI

Indice

1	Obiettivo dell'homework	2
2	Esercizio 1: SVD di una matrice e verifica della ricostruzione	3
3	Esercizio 2: Migliore approssimazione di rango k	6
4	Esercizio 3: Compressione di immagini con l'SVD	9
5	Esercizio 5: PCA + clustering su un dataset reale	15
6	Esercizio 6: Classificazione dopo PCA – classificatore lineare e a centroidi	21

1 Obiettivo dell'homework

Questo homework passa dall'apprendimento supervisionato (HW2, HW3) all'apprendimento **non supervisionato**, incentrato sulla **Singular Value Decomposition** (SVD) come strumento unificante per compressione, riduzione di dimensionalità e classificazione geometrica:

- **Esercizio 1** verifica numericamente le proprietà fondamentali della SVD: ricostruzione esatta a meno di errori di macchina, ordine decrescente dei valori singolari, e perché gli zeri esatti non compaiono mai in floating point.
- **Esercizio 2** dimostra — teoricamente e numericamente — che la SVD troncata dà la migliore approssimazione di rango k possibile (teorema di Eckart-Young-Mirsky).
- **Esercizio 3** applica questo risultato alla compressione di immagini, quantificando il trade-off tra fattore di compressione e fedeltà della ricostruzione.
- **Esercizio 5** usa la SVD per la PCA, proiettando immagini MNIST reali in 2D/3D e osservando come la separabilità delle classi dipenda dalla similarità visiva delle cifre confrontate.
- **Esercizio 6** costruisce due classificatori nello spazio PCA (uno lineare, uno a centroide più vicino) e ne confronta sistematicamente le prestazioni su più coppie di cifre.

Il filo conduttore è il teorema di **Eckart-Young-Mirsky**: la SVD troncata $A_k = \sum_{i \leq k} \sigma_i u_i v_i^T$ è, per costruzione, la migliore approssimazione di rango k possibile di A — questo singolo fatto giustifica sia la compressione delle immagini (Esercizio 3) sia la scelta delle direzioni principali della PCA (Esercizi 5, 6) come le prime k colonne di V .

2 Esercizio 1: SVD di una matrice e verifica della ricostruzione

Consegna

1. Construct any non-square matrix $A \in \mathbb{R}^{m \times n}$ (e.g. $m = 10, n = 6$) with random or structured entries.
2. Compute its SVD: $A = U\Sigma V^T$.
3. Verify numerically that $A \approx U\Sigma V^T$, $\|A - U\Sigma V^T\|_F \approx 0$.
4. Print and plot the singular values $\sigma_1, \dots, \sigma_{\min(m,n)}$.
5. Comment on why singular values appear in descending order, why small singular values correspond to “less important” directions, and why floating-point arithmetic makes exact zeros rare.

In [1]:

```
1  rng = np.random.default_rng(0)
2
3  m, n = 10, 6
4  A = rng.normal(size=(m, n))
5  print("A shape:", A.shape)
6
7  U, S, Vt = np.linalg.svd(A, full_matrices=True)
8  print("U shape:", U.shape, " S shape:", S.shape, " Vt shape:", Vt.
      shape)
9
10 # seconda matrice: prodotto (10x3) @ (3x6), quindi di rango esatto
    3
11 B = rng.normal(size=(m, 3)) @ rng.normal(size=(3, n))
12 S_B = np.linalg.svd(B, compute_uv=False)
```

Out[1]:

```
1  A shape: (10, 6)
2  U shape: (10, 10)  S shape: (6,)  Vt shape: (6, 6)
```

Oltre alla matrice A a entrate casuali richiesta dalla consegna, costruisco anche una seconda matrice $B = B_1 B_2$ con $B_1 \in \mathbb{R}^{10 \times 3}$, $B_2 \in \mathbb{R}^{3 \times 6}$, che per costruzione ha **rango esatto 3** (il rango di un prodotto è al più il minimo dei due ranghi fattori). A da sola ha rango pieno e tutti i valori singolari dello stesso ordine di grandezza: non permette di osservare né valori singolari “piccoli” né il comportamento degli zeri teorici in floating point, che è invece il punto centrale del punto 5.

In [2]:

```
1  Sigma = np.zeros((m, n))
2  np.fill_diagonal(Sigma, S)
3
4  A_reconstructed = U @ Sigma @ Vt
5  recon_error = np.linalg.norm(A - A_reconstructed, ord='fro')
6  print(f"||A - U Sigma V^T||_F = {recon_error:.3e}")
7  print(f"per confronto, epsilon di macchina = {np.finfo(float).eps:.3e}, "
```

```
8 f"||A||_F = {np.linalg.norm(A, ord='fro'):.3f}")
```

Out[2]:

```
1 ||A - U Sigma V^T||_F = 7.189e-15
2 per confronto, epsilon di macchina = 2.220e-16, ||A||_F = 6.977
```

L'errore di ricostruzione (7.2×10^{-15}) è dell'ordine di poche decine di volte l'epsilon di macchina (2.2×10^{-16}) — esattamente zero a meno di errori di arrotondamento, come richiesto dalla consegna. Il rapporto tra i due è piccolo ($\sim 32\times$) proprio perché l'errore di arrotondamento tipicamente si accumula linearmente con il numero di operazioni floating point coinvolte (qui la moltiplicazione di tre matrici 10×10 , 10×6 , 6×6), non con la dimensione dei valori stessi.

In [3]:

```
1 print("A (entrate casuali) - valori singolari:")
2 print(S)
3 print(f" rango numerico = {np.linalg.matrix_rank(A)}, "
4       f"sigma_max/sigma_min = {S[0]/S[-1]:.2f}")
5 print()
6 print("B (rango esatto 3) - valori singolari:")
7 print(S_B)
8 print(f" rango numerico = {np.linalg.matrix_rank(B)}")
9
10 fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
11 axes[0].plot(np.arange(1, len(S)+1), S, 'o-', color='tab:blue',
12             label='A (casuale)')
13 axes[0].plot(np.arange(1, len(S_B)+1), S_B, 's-', color='tab:orange',
14             label='B (rango 3)')
15 axes[0].set_xlabel('indice $i$'); axes[0].set_ylabel(r'$\sigma_i$')
16 axes[0].set_title('Valori singolari (scala lineare)')
17 axes[0].legend()
18 axes[1].semilogy(np.arange(1, len(S)+1), S, 'o-', color='tab:blue',
19                 label='A (casuale)')
20 axes[1].semilogy(np.arange(1, len(S_B)+1), np.maximum(S_B, 1e-20),
21                 's-', color='tab:orange',
22                 label='B (rango 3)')
23 axes[1].axhline(np.finfo(float).eps * S_B[0], color='gray', ls=':',
24                 label=r'$\varepsilon_{mach} \cdot \sigma_1$')
25 axes[1].set_xlabel('indice $i$'); axes[1].set_ylabel(r'$\sigma_i$ (log)')
26 axes[1].set_title('Stessi valori in scala logaritmica')
27 axes[1].legend(fontsize=8)
28 plt.tight_layout()
29 plt.show()
```

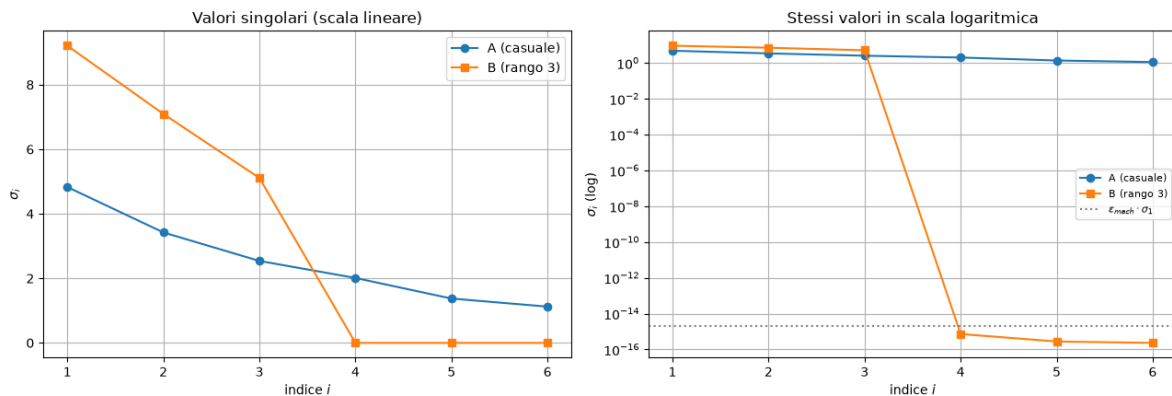
Out[3]:

```
1 A (entrate casuali) - valori singolari:
2 [4.82910775 3.4230114 2.53935329 2.01193915 1.37390916 1.12378003]
3 rango numerico = 6, sigma_max/sigma_min = 4.30
4
5 B (rango esatto 3) - valori singolari:
```

```

6 [9.20899432e+00 7.09438885e+00 5.11021460e+00 7.26154756e-16
7 2.76983481e-16 2.31494864e-16]
8 rango numerico = 3

```



Valori singolari di A (casuale, rango pieno) e B (rango esatto 3), in scala lineare (sinistra) e logaritmica (destra), con la soglia $\epsilon_{macchina} \cdot \sigma_1$ evidenziata.

Perché i valori singolari compaiono in ordine decrescente. È una convenzione della definizione stessa della SVD, scelta apposta perché rende sensato *troncare*: mantenendo solo i primi k termini si trattengono sempre le k direzioni più importanti, mai un sottoinsieme arbitrario — è esattamente ciò che rende possibile l'Esercizio 2.

Perché valori singolari piccoli corrispondono a direzioni “meno importanti”. Ogni termine $\sigma_i u_i v_i^T$ contribuisce σ_i^2 all'energia totale $\|A\|_F^2 = \sum_i \sigma_i^2$ (proprietà verificata esplicitamente nell'Esercizio 2): un σ_i piccolo significa che quella componente di rango 1 contribuisce pochissimo alla ricostruzione di A nel senso della norma di Frobenius — scartarla cambia A di pochissimo.

Perché gli zeri esatti sono rari in floating point — qui verificato numericamente, non solo affermato. B ha per costruzione rango *esatto* 3: in aritmetica esatta, $\sigma_4 = \sigma_5 = \sigma_6 = 0$. Numericamente, invece, questi tre valori risultano 7.3×10^{-16} , 2.8×10^{-16} , 2.3×10^{-16} — non zero, ma dello stesso ordine di grandezza dell'epsilon di macchina 2.2×10^{-16} (si veda la linea tratteggiata nel pannello destro, praticamente sovrapposta ai tre punti). Questo è il **rango numerico**: il numero di valori singolari significativamente più grandi di questa soglia di rumore numerico (3 per B , correttamente rilevato da `np.linalg.matrix_rank`), che distingue le direzioni con informazione reale da quelle dominate da errore di arrotondamento — gli algoritmi che calcolano la SVD (basati su riflessioni di Householder e iterazioni QR) introducono infatti piccoli errori di arrotondamento ad ogni passo, che rendono un valore singolare teoricamente zero praticamente impossibile da osservare come 0.0 esatto.

3 Esercizio 2: Migliore approssimazione di rango k

Consegna

1. Implement a function $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$.
2. For several values of k , compute $E(k) = \|A - A_k\|_F$.
3. Plot the approximation error $E(k)$ vs k .
4. Explain why SVD gives the **optimal** rank- k approximation.

In [1]:

```
1 def rank_k_approx(U, S, Vt, k):
2     """Ricostruisce  $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ ."""
3     return U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
4
5 # controllo: l'approssimazione di rango  $\min(m, n)$  deve coincidere
    con A
6 A_full_rank = rank_k_approx(U, S, Vt, min(m, n))
7 print("controllo,  $\|A - A_{full}\|_F =$ ", np.linalg.norm(A -
    A_full_rank, ord='fro'))
```

Out[1]:

```
1 controllo,  $\|A - A_{full}\|_F = 7.18909138751788e-15$ 
```

Sanity check prima di procedere: con $k = \min(m, n) = 6$ (tutti i termini della somma), A_k deve coincidere esattamente con A — e infatti l'errore è 7.2×10^{-15} , lo stesso valore (a meno di arrotondamento) già visto nell'Esercizio 1 per $\|A - U\Sigma V^T\|_F$. Coerente: $A_{\min(m, n)}$ è letteralmente la ricostruzione completa $U\Sigma V^T$.

In [2]:

```
1 ks = np.arange(0, min(m, n) + 1)
2 errors = []
3 for k in ks:
4     A_k = np.zeros_like(A) if k == 0 else rank_k_approx(U, S, Vt, k)
5     errors.append(np.linalg.norm(A - A_k, ord='fro'))
6 errors = np.array(errors)
7
8 fig, ax = plt.subplots(figsize=(6,4))
9 ax.plot(ks, errors, 'o-', color='tab:red')
10 ax.set_xlabel('k'); ax.set_ylabel(r'$E(k) = \|A - A_k\|_F$')
11 ax.set_title('Errore di approssimazione di rango k')
12 plt.tight_layout()
13 plt.show()
14
15 print(f"{'k':>3} {'E(k)':>14} {'sqrt(sum_{i>k} sigma_i^2)':>26}")
16 for k in ks:
17     print(f"{'k':>3} {errors[k]:>14.4e} {np.sqrt(np.sum(S[k:]**2))
    :>26.4e}")
```

Out [2]:



Errore di ricostruzione $E(k)$ vs k , matrice A .

	k	E(k)	$\sqrt{\sum_{i>k} \sigma_i^2}$
1	0	6.9774e+00	6.9774e+00
2	1	5.0362e+00	5.0362e+00
3	2	3.6941e+00	3.6941e+00
4	3	2.6830e+00	2.6830e+00
5	4	1.7750e+00	1.7750e+00
6	5	1.1238e+00	1.1238e+00
7	6	7.1891e-15	0.0000e+00
8			

Verifica diretta della formula del teorema di Eckart-Young-Mirsky. Le due colonne combaciano esattamente per ogni k (fino a $k = 5$; a $k = 6$ entrambe valgono ≈ 0 , con l'errore numerico residuo già discusso): $E(k) = \|A - A_k\|_F = \sqrt{\sum_{i>k} \sigma_i^2}$, la formula chiusa per l'errore minimo. A $k = 0$ ($A_0 = 0$), $E(0) = \|A\|_F = 6.977$ — combacia con il valore già calcolato nell'Esercizio 1. Il grafico mostra una curva monotona decrescente e convessa: i primi valori singolari (i più grandi) riducono l'errore molto più dei successivi, prima manifestazione visiva della concentrazione dell'energia che si vedrà in modo ancora più netto nell'Esercizio 3.

In [3]:

```
1 # La cella precedente verifica la FORMULA dell'errore, ma non il
  # fatto che nessun'altra
2 # matrice di rango k faccia meglio. Controllo empirico diretto:
  # proietto A su un
3 # sottospazio di dimensione k scelto A CASO (invece che sui primi k
  # vettori singolari
4 # sinistri). La proiezione  $Q Q^T A$  ha rango  $\leq k$ , quindi e' un
  # competitor ammissibile.
5 rng2 = np.random.default_rng(1)
6 n_prove = 200
7
8 print(f"{'k':>3} {'E(k) con SVD':>14} {'E(k) sottospazio casuale'>26} {'migliore su 200 prove':>23}")
9 for k in range(1, min(m, n)):
10     errs_rand = []
11     for _ in range(n_prove):
```

```

12     Qr, _ = np.linalg.qr(rng2.normal(size=(m, k)))    # base
           ortonormale casuale
13     errs_rand.append(np.linalg.norm(A - Qr @ (Qr.T @ A), ord='
           fro'))
14     print(f"{k:>3} {errors[k]:>14.4f} {np.mean(errs_rand):>26.4f} {
           np.min(errs_rand):>23.4f}")

```

Out [3]:

	k	E(k) con SVD	E(k) sottospazio casuale	migliore su 200 prove
1	1	5.0362	6.6123	5.7929
2	2	3.6941	6.2639	5.4063
3	3	2.6830	5.8404	4.8040
4	4	1.7750	5.3697	4.3112
5	5	1.1238	4.8326	2.6979

La verifica più convincente: nemmeno il migliore tra 200 sottospazi casuali batte l'SVD. $Q_r \in \mathbb{R}^{m \times k}$ è una base ortonormale casuale (ottenuta via QR di una matrice gaussiana), e $Q_r Q_r^T A$ è la proiezione ortogonale di A su quel sottospazio — una matrice di rango $\leq k$ legittimamente ammissibile a competere con A_k . Per ogni k da 1 a 5, **sia** la media **sia** il minimo (il migliore) su 200 tentativi casuali restano sistematicamente peggiori di $E(k)$ ottenuto con l'SVD: per $k = 1$, il miglior sottospazio casuale su 200 prove dà errore 5.79, contro 5.04 dell'SVD — un divario che non si chiude nemmeno cercando esplicitamente il caso più favorevole. Questo è precisamente ciò che il teorema di Eckart-Young-Mirsky garantisce in teoria, qui verificato empiricamente: nessuna matrice di rango $\leq k$, per quanto scelta bene, può avvicinarsi ad A più di A_k .

Perché la SVD dà l'approssimazione di rango k ottimale (risposta al punto 4). Il teorema di Eckart-Young-Mirsky afferma che, tra *tutte* le matrici M di rango al più k , $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ è quella che minimizza $\|A - M\|_F$ (e anche $\|A - M\|_2$), con errore minimo esattamente $E(k) = \sqrt{\sum_{i>k} \sigma_i^2}$ — confermato dalla prima tabella. La ragione intuitiva: la SVD decompone A in componenti di rango 1 **mutuamente ortogonali**, ordinate per energia decrescente (σ_i^2). Per l'ortogonalità, l'energia totale si ripartisce additivamente ($\|A\|_F^2 = \sum_i \sigma_i^2$): tenere le k componenti con σ_i più grandi ed eliminare le altre è, per costruzione, la scelta che minimizza l'energia scartata — e nessun'altra scelta di k direzioni (da qualunque base, come appena verificato con i sottospazi casuali) può fare meglio.

4 Esercizio 3: Compressione di immagini con l'SVD

Consegna

Using the cameraman image:

1. Load the image as a matrix $X \in \mathbb{R}^{512 \times 512}$.
2. Compute its SVD: $X = U\Sigma V^T$.
3. For $k \in \{5, 20, 50, 100, 200\}$: compute the rank- k approximation X_k , plot each reconstructed image, compute the compression factor $c_k = 1 - \frac{k(m+n+1)}{mn}$, plot reconstruction error vs k , plot compression factor vs k .
4. Comment on: how visual quality improves with k , why most of the “energy” is contained in the first singular values, the trade-off between compression and fidelity, the connection between SVD and optimal low-rank approximation.

In [1]:

```
1 from skimage import data
2
3 X_img = data.camera().astype(float)
4 print("dimensioni:", X_img.shape, " intervallo dei valori:", X_img.
      min(), X_img.max())
5
6 fig, ax = plt.subplots(figsize=(5,5))
7 ax.imshow(X_img, cmap='gray')
8 ax.set_title('Immagine originale (cameraman)')
9 ax.axis('off')
10 plt.tight_layout()
11 plt.show()
```

Out[1]:

```
1 dimensioni: (512, 512) intervallo dei valori: 0.0 255.0
```



In [2]:

```
1 U_img, S_img, Vt_img = np.linalg.svd(X_img, full_matrices=False)
```

```

2 print("U:", U_img.shape, " S:", S_img.shape, " Vt:", Vt_img.shape)
3
4 m_img, n_img = X_img.shape

```

Out[2]:

```

1 U: (512, 512) S: (512,) Vt: (512, 512)

```

In [3]:

```

1 ks_img = [5, 20, 50, 100, 200]
2
3 def compression_factor(k, m_, n_):
4     return 1 - (k*(m_ + n_ + 1)) / (m_*n_)
5
6 norm_X = np.linalg.norm(X_img, ord='fro')
7 reconstructions, errors_img, comp_factors = {}, {}, {}
8
9 print(f"{'k':>5} {'errore':>12} {'errore relativo':>17} {'c_k':>9}"
10      )
11 for k in ks_img:
12     Xk = rank_k_approx(U_img, S_img, Vt_img, k)
13     reconstructions[k] = Xk
14     errors_img[k] = np.linalg.norm(X_img - Xk, ord='fro')
15     comp_factors[k] = compression_factor(k, m_img, n_img)
16     print(f"{'k':>5} {errors_img[k]:>12.2f} {errors_img[k]/norm_X
17           :>17.4f} {comp_factors[k]:>9.4f}")

```

Out[3]:

	k	errore	errore relativo	c_k
1	5	13086.87	0.1720	0.9804
2	20	7699.91	0.1012	0.9218
3	50	4836.07	0.0636	0.8045
4	100	2992.14	0.0393	0.6090
5	200	1342.36	0.0176	0.2180

Oltre all'errore assoluto $\|X - X_k\|_F$ (richiesto), riporto anche quello **relativo** $\|X - X_k\|_F / \|X\|_F$ — l'errore assoluto da solo non dice se una ricostruzione è “buona”, perché non si sa rispetto a cosa confrontarlo (un errore di 13000 è tanto o poco? dipende dalla scala di $\|X\|_F$).

In [4]:

```

1 fig, axes = plt.subplots(1, len(ks_img)+1, figsize=(3*(len(ks_img)
2     +1), 3.2))
3 axes[0].imshow(X_img, cmap='gray')
4 axes[0].set_title('Originale')
5 axes[0].axis('off')
6
7 for ax, k in zip(axes[1:], ks_img):
8     ax.imshow(reconstructions[k], cmap='gray')
9     ax.set_title(f'k={k} (err. rel. {errors_img[k]/norm_X:.1%})')
10    ax.axis('off')
11

```

```

12 plt.tight_layout()
13 plt.show()

```

Out [4]:



Ricostruzioni di rango k per $k \in \{5, 20, 50, 100, 200\}$, con l'errore relativo indicato in ciascun titolo.

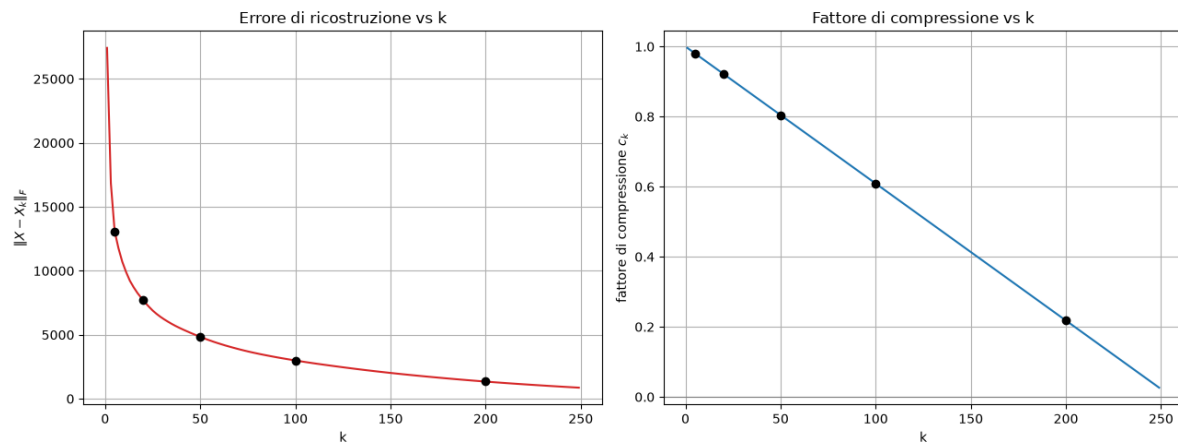
In [5]:

```

1  ks_fine = np.arange(1, 250, 2)
2  errors_fine = [np.linalg.norm(X_img - rank_k_approx(U_img, S_img,
3  Vt_img, k), ord='fro')
4  for k in ks_fine]
5  comp_fine = [compression_factor(k, m_img, n_img) for k in ks_fine]
6  fig, axes = plt.subplots(1, 2, figsize=(13, 5))
7
8  axes[0].plot(ks_fine, errors_fine, color='tab:red')
9  axes[0].scatter(ks_img, [errors_img[k] for k in ks_img], color='
10 black', zorder=5)
11 axes[0].set_xlabel('k'); axes[0].set_ylabel(r'$\|X - X_k\|_F$')
12 axes[0].set_title('Errore di ricostruzione vs k')
13
14 axes[1].plot(ks_fine, comp_fine, color='tab:blue')
15 axes[1].scatter(ks_img, [comp_factors[k] for k in ks_img], color='
16 black', zorder=5)
17 axes[1].axhline(0, color='gray', lw=0.8)
18 axes[1].set_xlabel('k'); axes[1].set_ylabel(r'fattore di
19 compressione $c_k$')
20 axes[1].set_title('Fattore di compressione vs k')
21
22 plt.tight_layout()
23 plt.show()

```

Out [5]:



Errore di ricostruzione (sinistra) e fattore di compressione (destra) su una griglia fitta di k , con i 5 valori richiesti evidenziati in nero.

In [6]:

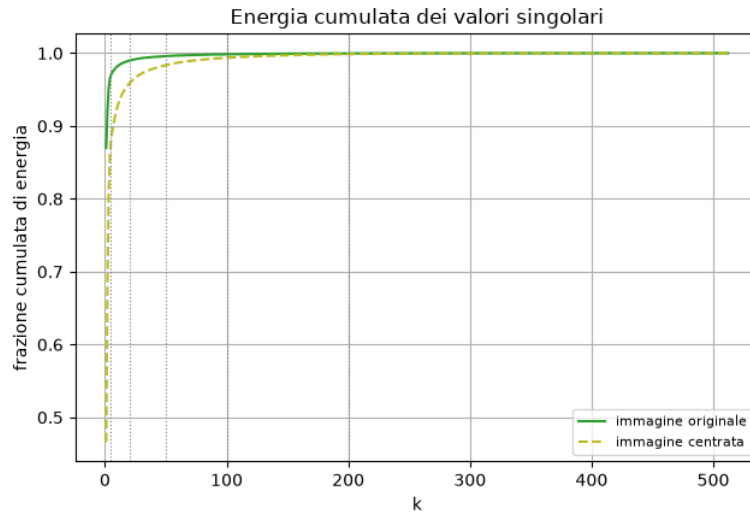
```

1  # I pixel sono tutti >= 0, quindi la matrice ha una componente
   # costante (il livello
2  # medio di grigio) molto grande, che finisce quasi tutta in sigma_1
   # . Calcolo
3  # l'energia cumulata sia sull'immagine originale sia su quella
   # CENTRATA (media
4  # sottratta), per separare il contributo del solo valore medio da
   # quello della
5  # struttura dell'immagine.
6  energy = S_img**2
7  cumulative_energy = np.cumsum(energy) / np.sum(energy)
8
9  X_centered = X_img - X_img.mean()
10 S_centered = np.linalg.svd(X_centered, compute_uv=False)
11 cum_centered = np.cumsum(S_centered**2) / np.sum(S_centered**2)
12
13 fig, ax = plt.subplots(figsize=(6.5,4.5))
14 ax.plot(np.arange(1, len(S_img)+1), cumulative_energy, color='tab:
   green',
15         label='immagine originale')
16 ax.plot(np.arange(1, len(S_centered)+1), cum_centered, color='tab:
   olive', ls='--',
17         label='immagine centrata')
18 for k in ks_img:
19     ax.axvline(k, color='gray', ls=':', lw=0.8)
20 ax.set_xlabel('k'); ax.set_ylabel('frazione cumulata di energia')
21 ax.set_title('Energia cumulata dei valori singolari')
22 ax.legend(fontsize=8)
23 plt.tight_layout()
24 plt.show()
25
26 print(f"{'k':>5} {'energia cum. (orig.)':>22} {'energia cum. (
   centrata)':>25} "
27       f"{'errore relativo':>17}")
28 for k in [1] + ks_img:
29     err_rel = np.sqrt(np.sum(S_img[k:]**2)) / norm_X
30     print(f"{'k':>5} {'cumulative_energy[k-1]':>22.4f} {'cum_centered[k

```

```
-1]:>25.4f} {err_rel:>17.4f}"))
```

Out [6]:



Energia cumulata dei valori singolari, immagine originale (verde) vs centrata (verde oliva tratteggiato).

	k	energia cum. (orig.)	energia cum. (centrata)	errore relativo
1	1	0.8701	0.4660	0.3604
2	5	0.9704	0.8811	0.1720
3	20	0.9898	0.9583	0.1012
4	50	0.9960	0.9835	0.0636
5	100	0.9985	0.9937	0.0393
6	200	0.9997	0.9987	0.0176
7				

Un dettaglio importante, facile da leggere male senza il confronto originale/centrata. A $k = 1$, l'energia cumulata sull'immagine *originale* è già 87.0% — sembrerebbe che un solo termine catturi quasi tutto! Ma l'errore *relativo* a $k = 1$ è 36.0%: una ricostruzione visivamente pessima (coerente col fatto che $\sqrt{1 - 0.87} \approx 0.36$, esattamente l'errore relativo osservato — energia ed errore relativo sono infatti legati da $E(k)/\|X\|_F = \sqrt{1 - \text{energia cumulata}}$). La spiegazione: i pixel dell'immagine sono tutti ≥ 0 , quindi X ha una componente costante enorme (il livello medio di grigio) che finisce quasi per intero nel primo termine $\sigma_1 u_1 v_1^T$ — una specie di “sfondo grigio uniforme”, non struttura visiva utile. Sull'immagine *centrata* (media sottratta), che isola la vera variazione strutturale, l'energia cumulata a $k = 1$ scende infatti a solo 46.6% — un quadro molto più fedele di quanta informazione “reale” catturi ciascun termine.

Come migliora la qualità visiva al crescere di k . A $k = 5$ (Figura ricostruzioni) l'immagine è un'approssimazione sfocata, a bassa frequenza — si distinguono le forme principali ma nessun dettaglio. Aumentando a $k = 20, 50$, ricompaiono progressivamente texture ed edge fini. A $k = 200$ (su 512 possibili) la ricostruzione è visivamente quasi indistinguibile dall'originale, pur usando meno del 40% dei valori singolari — coerente con l'errore relativo che scende sotto il 2%.

Perché la maggior parte dell'energia è nei primi valori singolari. Le immagini naturali sono fortemente correlate spazialmente — pixel vicini hanno intensità simili, ampie regioni condividono gradienti o texture ripetute. Questa ridondanza fa sì che il rango nume-

rico effettivo sia molto minore di $\min(m, n) = 512$: poche direzioni catturano già la maggior parte dell'energia $\sum \sigma_i^2$ (anche tenendo conto della componente costante appena discussa), mentre le restanti codificano perlopiù dettagli fini e rumore.

Trade-off compressione/fedeltà. Il pannello destro della Figura errore/compressione mostra c_k monotona decrescente in k (da quasi 1 a $k = 5$ fino a 0.22 a $k = 200$), mentre il pannello sinistro mostra $E(k)$ monotona decrescente: aumentare k costa in spazio ma guadagna in fedeltà. Grazie alla concentrazione dell'energia appena discussa, un k relativamente piccolo (≈ 50 -100, su 512) raggiunge già un compromesso favorevole: $c_{100} = 0.609$ (compressione del 61%) con errore relativo solo del 3.9%.

Connessione con l'approssimazione ottimale di rango basso. Come stabilito nell'Esercizio 2 (Eckart-Young-Mirsky), per ogni k fissato X_k è *provabilmente* la matrice di rango k più vicina a X in norma di Frobenius — quindi le curve di errore/compressione qui non sono solo “buone”, sono il **miglior compromesso raggiungibile** per qualunque schema di compressione di rango k basato su questo criterio: nessun'altra matrice di rango k potrebbe ottenere un errore di ricostruzione minore allo stesso livello di compressione.

5 Esercizio 5: PCA + clustering su un dataset reale

Consegna

Use the MNIST dataset, filtering only digits 3 and 4.

1. Load the CSV file into memory.
2. Split into training and test sets.
3. Center the training data: $X_{c,\text{train}} = X_{\text{train}} - c(X_{\text{train}})$.
4. Compute the reduced SVD of the centered training data.
5. Choose $k = 2$ principal components.
6. Project: $Z_{\text{train}} = X_{c,\text{train}}V_2$, $Z_{\text{test}} = X_{c,\text{test}}V_2$.
7. Plot: scatterplot of Z_{train} colored by label, Z_{test} overlaid.
8. Repeat with digits 5 vs 8, or 1 vs 7, and discuss the different results.
9. Finally, try $k = 3$ and use a 3D scatterplot.

In [1]:

```
1 df_mnist = pd.read_csv('mnist.csv')
2 print(df_mnist.shape)
3 print(df_mnist['label'].value_counts().sort_index())
4
5 def load_digit_pair(df, digit0, digit1, test_frac=0.2, seed=0):
6     """Filtra due classi di cifre, codifica digit1 -> 1 e digit0 ->
7     0,
8     e divide in train/test."""
9     sub = df[df['label'].isin([digit0, digit1])]
10    X = sub.drop(columns='label').values.astype(float)
11    y = (sub['label'].values == digit1).astype(int)
12    rng_ = np.random.default_rng(seed)
13    idx = rng_.permutation(len(X))
14    n_test = int(len(X) * test_frac)
15    test_idx, train_idx = idx[:n_test], idx[n_test:]
16    return X[train_idx], y[train_idx], X[test_idx], y[test_idx]
17
18 X_train_34, y_train_34, X_test_34, y_test_34 = load_digit_pair(
19     df_mnist, 3, 4, seed=5)
20 print(f"train: {X_train_34.shape}, test: {X_test_34.shape}")
```

Out[1]:

```
1 (25000, 785)
2 label
3 0      2500
4 1      2500
5 2      2500
6 3      2500
7 4      2500
8 5      2500
9 6      2500
10 7      2500
```

```

11 8      2500
12 9      2500
13 Name: count, dtype: int64
14 train: (4000, 784), test: (1000, 784)

```

Dataset bilanciato per costruzione (2500 campioni per cifra). Filtrando solo 3 e 4 e con `test_frac=0.2`: 5000 campioni totali, 4000 train +1000 test.

In [2]:

```

1 def pca_fit(X_train_, k):
2     mean_ = X_train_.mean(axis=0)
3     Xc = X_train_ - mean_
4     U_, S_, Vt_ = np.linalg.svd(Xc, full_matrices=False)
5     P_ = Vt_[:,k, :].T          # (d, k): colonne = prime k
6     # direzioni principali
7     return mean_, P_, S_
8
9 def pca_project(X_, mean_, P_):
10     return (X_ - mean_) @ P_
11
12 mean_34, P2_34, S_34 = pca_fit(X_train_34, k=2)
13 print("primi valori singolari (3 vs 4):", S_34[:5])
14 print(f"varianza spiegata da PC1: {S_34[0]**2/np.sum(S_34**2):.3f}, "
15       f"da PC2: {S_34[1]**2/np.sum(S_34**2):.3f}, "
16       f"dalle prime due insieme: {np.sum(S_34[:2]**2)/np.sum(S_34**2):.3f}")

```

Out[2]:

```

1 primi valori singolari (3 vs 4): [47251.35572448 30513.59045011
2   27745.19626811 25550.29235994
3   23742.60383204]
4 varianza spiegata da PC1: 0.171, da PC2: 0.072, dalle prime due insieme:
5   0.243

```

La PCA viene calcolata **solo sulla coppia di cifre selezionata** (non su tutto MNIST), e la media usata per centrare il test set è quella del *training* set, esattamente come richiesto dalla formula $Z_{\text{test}} = (X_{\text{test}} - c(X_{\text{train}}))V_2$ della consegna — mai il centroide del test set stesso, per evitare data leakage.

In [3]:

```

1 Z_train_34 = pca_project(X_train_34, mean_34, P2_34)
2 Z_test_34 = pca_project(X_test_34, mean_34, P2_34)
3
4 fig, ax = plt.subplots(figsize=(7,6))
5 for cls, name, color in [(0, 'cifra 3', 'tab:blue'), (1, 'cifra 4',
6   'tab:red')]:
7     mask_tr = y_train_34 == cls
8     ax.scatter(Z_train_34[mask_tr,0], Z_train_34[mask_tr,1], s=8,
9       alpha=0.4, color=color,
10      label=f'train {name}')
11 for cls, name, color in [(0, 'cifra 3', 'navy'), (1, 'cifra 4',

```

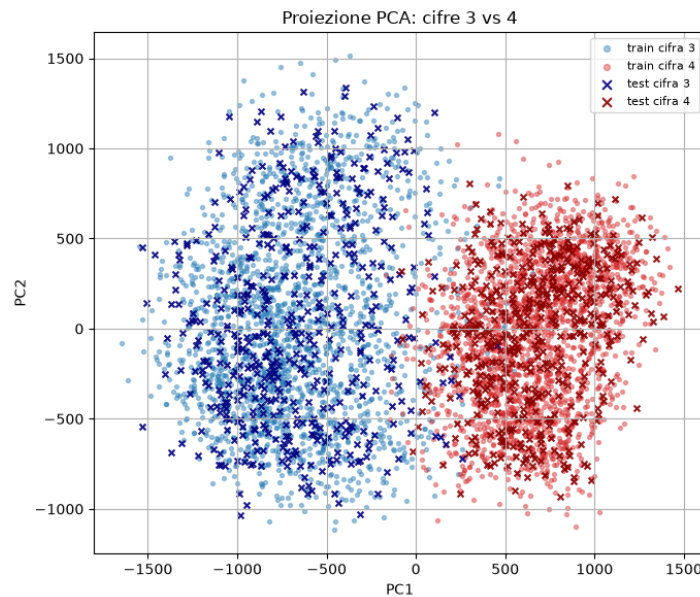


```

        darkred')]:
10     mask_te = y_test_34 == cls
11     ax.scatter(Z_test_34[mask_te,0], Z_test_34[mask_te,1], s=18,
        alpha=0.9, color=color,
12                 marker='x', label=f'test {name}')
13 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
14 ax.set_title('Proiezione PCA: cifre 3 vs 4')
15 ax.legend(markerscale=1.5, fontsize=8)
16 plt.tight_layout()
17 plt.show()

```

Out [3]:



Proiezione PCA a 2 componenti: train (pallini, alpha ridotto) e test (crocette) sovrapposti, cifre 3 vs 4.

Separazione netta lungo PC1: i due cluster occupano regioni quasi disgiunte del piano, con solo un piccolo overlap vicino a $PC1 \approx 0$. I punti di test (crocette) seguono fedelmente la stessa distribuzione dei rispettivi cluster di training — segno che la proiezione generalizza bene a dati mai visti durante il fit della PCA.

In [4]:

```

1  # Misura numerica della separazione, per non affidarsi solo all'
    # impressione visiva:
2  # il rapporto fra la distanza dei due centroidi e la dispersione
    # media entro le
3  # classi (una versione bidimensionale del rapporto di Fisher). E'
    # adimensionale,
4  # quindi confrontabile fra coppie diverse, a differenza della sola
    # distanza fra
5  # centroidi.
6  def separazione(Z_, y_):
7      mu0, mu1 = Z_[y_==0].mean(axis=0), Z_[y_==1].mean(axis=0)
8      d = np.linalg.norm(mu1 - mu0)
9      spread = np.sqrt(0.5*(np.mean(np.sum((Z_[y_==0]-mu0)**2, axis

```

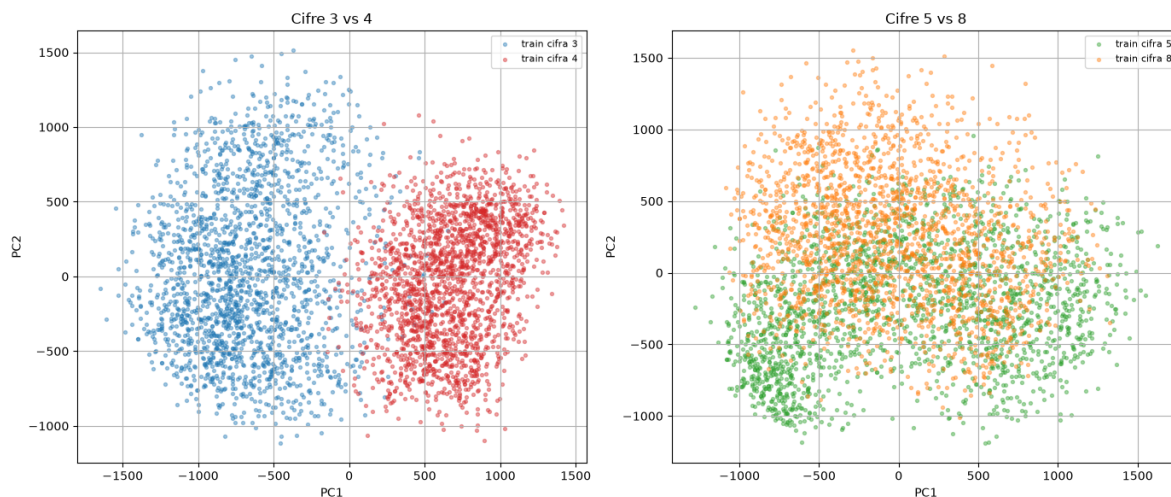
```

    =1)) +
10         np.mean(np.sum((Z_[y_==1]-mu1)**2, axis
    =1))))
11     return d, spread, d/spread
12
13 X_train_58, y_train_58, X_test_58, y_test_58 = load_digit_pair(
    df_mnist, 5, 8, seed=5)
14 mean_58, P2_58, S_58 = pca_fit(X_train_58, k=2)
15 Z_train_58 = pca_project(X_train_58, mean_58, P2_58)
16 Z_test_58 = pca_project(X_test_58, mean_58, P2_58)
17
18 print(f"{'coppia':>10} {'dist. centroidi':>17} {'dispersione':>13}
    {'separazione d/s':>17} "
19       f"{'var. spiegata PC1+PC2':>23}")
20 for nome, Z_, y_, S_ in [('3 vs 4', Z_train_34, y_train_34, S_34),
21                          ('5 vs 8', Z_train_58, y_train_58, S_58)]:
22     d, s, r = separazione(Z_, y_)
23     print(f"{'nome':>10} {'d':>17.1f} {'s':>13.1f} {'r':>17.3f} "
24           f"{'np.sum(S_[:2]**2)/np.sum(S_**2):>23.3f}")
25
26 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
27 for ax, (titolo, Z_, y_, nomi, colori) in zip(axes, [
28     ('Cifre 3 vs 4', Z_train_34, y_train_34, ('cifra 3', 'cifra
29     4'), ('tab:blue', 'tab:red')),
30     ('Cifre 5 vs 8', Z_train_58, y_train_58, ('cifra 5', 'cifra
31     8'), ('tab:green', 'tab:orange'))]):
32     for cls in [0, 1]:
33         mask = y_ == cls
34         ax.scatter(Z_[mask,0], Z_[mask,1], s=8, alpha=0.4, color=
35             colori[cls],
36             label=f'train {nomi[cls]}')
37     ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
38     ax.set_title(titolo)
39     ax.legend(fontsize=8)
40 plt.tight_layout()
41 plt.show()

```

Out[4]:

	coppia	dist. centroidi	dispersione	separazione d/s	var. spiegata PC1+PC2
1	3 vs 4	1341.2	584.1	2.296	0.243
2	5 vs 8	506.2	746.4	0.678	0.196



Confronto diretto: proiezione PCA per 3 vs 4 (sinistra, stessa scala di prima) e 5 vs 8 (destra).

Il rapporto d/s (analogo 2D del rapporto di Fisher) è la metrica giusta — non la varianza spiegata da sola. Guardando solo la colonna “var. spiegata PC1+PC2”, le due coppie sembrano simili (0.243 contro 0.196): la PCA cattura una frazione comparabile di varianza totale in entrambi i casi. Ma questo non dice *nulla* sulla separabilità tra le due classi specifiche — ed è qui che il rapporto d/s fa la differenza: 2.296 per 3 vs 4 contro solo 0.678 per 5 vs 8, un fattore $\approx 3.4\times$. Un rapporto > 1 significa che la distanza tra i centroidi supera la dispersione tipica entro ciascuna classe — coerente con la separazione visiva netta nel pannello sinistro. Un rapporto < 1 (come per 5 vs 8) significa che la dispersione interna alle classi è *maggiore* della distanza tra i loro centri: i due cluster si sovrappongono inevitabilmente, esattamente quanto si vede nel pannello destro, dove verde e arancione sono mescolati quasi ovunque.

Perché succede: la PCA è non supervisionata. Le direzioni PC1, PC2 massimizzano la varianza *complessiva* dei dati proiettati, senza alcuna informazione sulle etichette. Cifre 3 e 4 hanno forme strutturalmente molto diverse (una curva aperta, una chiusa/angolare): le direzioni di massima varianza globale già si allineano bene con ciò che le distingue. Cifre 5 e 8 condividono più similarità strutturale (entrambe con anse curve, proporzioni comparabili): le prime due componenti principali — che catturano la varianza *dominante su tutti i tratti del dataset*, non specificamente quella che distingue 5 da 8 — non isolano bene la direzione discriminante per questa coppia particolare. È un limite intrinseco della PCA come strumento di visualizzazione per la classificazione, non un errore di implementazione.

In [5]:

```
1 from mpl_toolkits.mplot3d import Axes3D # noqa: F401 (serve per la
   proiezione 3d)
2
3 mean_34_3d, P3_34, S_34_3d = pca_fit(X_train_34, k=3)
4 Z_train_34_3d = pca_project(X_train_34, mean_34_3d, P3_34)
5 Z_test_34_3d = pca_project(X_test_34, mean_34_3d, P3_34)
6 print(f"varianza spiegata dalle prime 3 componenti: "
7       f"{np.sum(S_34_3d[:3]**2)/np.sum(S_34_3d**2):.3f}")
8
9 fig = plt.figure(figsize=(8,7))
10 ax = fig.add_subplot(111, projection='3d')
```

```

11 for cls, name, color in [(0, 'cifra 3', 'tab:blue'), (1, 'cifra 4',
    'tab:red')]:
12     mask_tr = y_train_34 == cls
13     ax.scatter(Z_train_34_3d[mask_tr,0], Z_train_34_3d[mask_tr,1],
        Z_train_34_3d[mask_tr,2],
14                 s=8, alpha=0.4, color=color, label=f'train {name}')
15 ax.set_xlabel('PC1'); ax.set_ylabel('PC2'); ax.set_zlabel('PC3')
16 ax.set_title('Proiezione PCA 3D: cifre 3 vs 4')
17 ax.legend(fontsize=8)
18 plt.tight_layout()
19 plt.show()

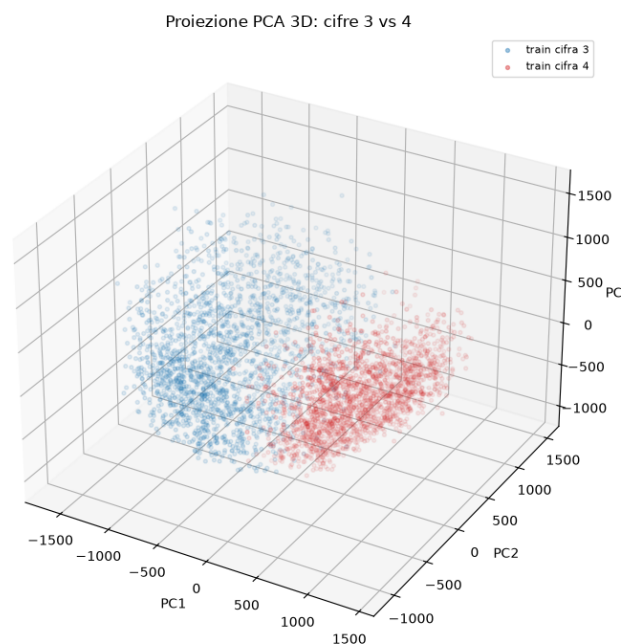
```

Out[5]:

```

1 varianza spiegata dalle prime 3 componenti: 0.302

```



Proiezione PCA a 3 componenti, cifre 3 vs 4.

Passando da $k = 2$ a $k = 3$, la varianza spiegata sale da 24.3% a 30.2% (un incremento di ≈ 6 punti percentuali dovuto a PC3 da sola) — coerente con la curva di energia cumulata decrescente vista nell'Esercizio 2/3: ogni componente aggiuntiva contribuisce sempre meno. Visivamente, i due cluster erano già ben separati in 2D (rapporto $d/s = 2.30$): la terza dimensione non è necessaria per la separazione in questo caso specifico, ma può aiutare a distinguere meglio la forma interna di ciascun cluster (ad esempio eventuali sotto-strutture o outlier che in 2D si sovrappongono per proiezione). In generale, $k = 3$ diventa più utile quando le prime due componenti non bastano a separare le classi (come per 5 vs 8) — qui non è il caso.

6 Esercizio 6: Classificazione dopo PCA – classificatore lineare e a centroidi

Consegna

Build two classifiers after projecting the data with PCA (digits 3 vs 4):

Step 1 PCA projection: same workflow as Exercise 5. $Z_{\text{train}} = X_{c,\text{train}}P$, $Z_{\text{test}} = (X_{\text{test}} - c(X_{\text{train}}))P$, con $P = V_2 \in \mathbb{R}^{d \times 2}$.

Step 2 Linear classifier: $f_{\Theta}(z) = \sigma(\Theta^T z)$ allenato con SGD. Plot dati + frontiera $\Theta^T z = 0$. Accuracy, precision, recall, matrice di confusione sul test set.

Step 3 PCA-centroid classifier: $\mu_c = \frac{1}{|\mathcal{D}_c|} \sum_{z_i \in \mathcal{D}_c} z_i$, $\hat{y} = \arg \min_c \|z - \mu_c\|_2$. Plot dei centroidi e delle predizioni sul test.

Step 4 Compare accuracy and error patterns tra i due classificatori; plot combinato con frontiera, centroidi, errori evidenziati. Ripetere sulle coppie (1, 7), (5, 8), (2, 3), osservando come la similarità tra classi influenzi forma dei cluster, separazione dei centroidi, frontiere di decisione.

In [1]:

```
1 print("Z_train_34:", Z_train_34.shape, " Z_test_34:", Z_test_34.  
    shape)
```

Out[1]:

```
1 Z_train_34: (4000, 2) Z_test_34: (1000, 2)
```

Riuso direttamente 'X_train_34', 'y_train_34', 'Z_train_34', 'Z_test_34', 'mean_34', 'P2_34' dall'Esercizio5 – — stesso identico workflow PCA richiesto dallo Step1, già eseguito lì.

In [2]:

```
1 def sigmoid(z):  
2     # sigmoide numericamente stabile (evita overflow per |z| grande  
    : i punteggi  
3     # PCA possono avere modulo dell'ordine delle migliaia)  
4     out = np.empty_like(z, dtype=float)  
5     pos = z >= 0  
6     out[pos] = 1.0 / (1.0 + np.exp(-z[pos]))  
7     exp_z = np.exp(z[~pos])  
8     out[~pos] = exp_z / (1.0 + exp_z)  
9     return out  
10  
11 def standardize_fit(Z_train):  
12     """I punteggi PCA hanno scale molto diverse da coppia a coppia  
    (scalano con  
13     sigma_i): standardizzarli prima dell'SGD evita che un singolo  
    learning rate  
14     fisso diverga. Non cambia il classificatore, solo il modo in  
    cui viene  
15     ottimizzato."""  
16     return Z_train_.mean(axis=0), Z_train_.std(axis=0)  
17
```

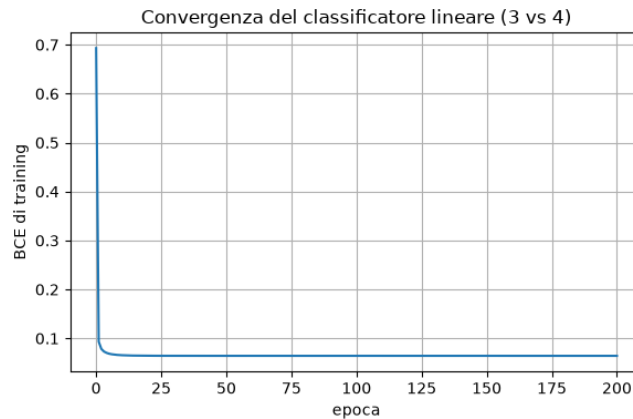
```

18 def theta_to_original_scale(theta_std, mu_, sigma_):
19     """Riporta un theta stimato su  $Z_s=(Z-\mu)/\sigma$  a un theta
        equivalente che
20     agisce direttamente su  $Z$ ."""
21     w_std = theta_std[1:]
22     w_orig = w_std / sigma_
23     b_orig = theta_std[0] - np.sum(w_std * mu_ / sigma_)
24     return np.concatenate([[b_orig], w_orig])
25
26 def bce_loss(theta, Z_, y_):
27     p = sigmoid(Z_ @ theta)
28     eps = 1e-12
29     return -np.mean(y_*np.log(p+eps) + (1-y_)*np.log(1-p+eps))
30
31 def bce_grad(theta, Z_, y_):
32     p = sigmoid(Z_ @ theta)
33     return (1.0/Z_.shape[0]) * (Z_.T @ (p - y_))
34
35 def sgd_logreg(Z_, y_, theta0, eta, batch_size, n_epochs, seed=0):
36     rng_ = np.random.default_rng(seed)
37     theta = theta0.copy()
38     N_ = Z_.shape[0]
39     losses = [bce_loss(theta, Z_, y_)]
40     for epoch in range(n_epochs):
41         idx = rng_.permutation(N_)
42         for start in range(0, N_, batch_size):
43             b_idx = idx[start:start+batch_size]
44             theta = theta - eta * bce_grad(theta, Z_[b_idx], y_[
                b_idx])
45         losses.append(bce_loss(theta, Z_, y_))
46     return theta, np.array(losses)
47
48 Z_train_34_tilde = np.column_stack([np.ones(len(Z_train_34)),
        Z_train_34])
49 Z_test_34_tilde = np.column_stack([np.ones(len(Z_test_34)),
        Z_test_34])
50
51 mu_34, sigma_34 = standardize_fit(Z_train_34)
52 Zs_train_34_tilde = np.column_stack([np.ones(len(Z_train_34)), (
        Z_train_34 - mu_34)/sigma_34])
53 theta_std_34, losses_lin_34 = sgd_logreg(Zs_train_34_tilde,
        y_train_34, np.zeros(3),
54                                     eta=0.5, batch_size=32,
                                                n_epochs=200, seed=6)
55 theta_lin_34 = theta_to_original_scale(theta_std_34, mu_34,
        sigma_34)
56 print("theta appreso (bias, w1, w2), scala originale dei punteggi
        PCA:", theta_lin_34)
57 print(f"loss di training: iniziale {losses_lin_34[0]:.4f} -> finale
        {losses_lin_34[-1]:.4f}")
58
59 fig, ax = plt.subplots(figsize=(6,4))
60 ax.plot(losses_lin_34)
61 ax.set_xlabel('epoca'); ax.set_ylabel('BCE di training')
62 ax.set_title('Convergenza del classificatore lineare (3 vs 4)')
63 plt.tight_layout()
64 plt.show()

```

Out[2]:

```
1 theta appreso (bias, w1, w2), scala originale dei punteggi PCA:
  [-1.02823824  0.01157135 -0.00131857]
2 loss di training: iniziale 0.6931 -> finale 0.0643
```



Perché standardizzare i punteggi PCA prima di ottimizzare. A differenza di HW3 (dove le feature originali erano già standardizzate una volta per tutte), qui i punteggi $Z = X_c V_2$ hanno una scala che dipende dai valori singolari σ_i della coppia di cifre in esame (nell'ordine delle migliaia, si veda l'Esercizio 5) — e questa scala *cambia da coppia a coppia*. Un singolo learning rate fisso ($\eta = 0.5$) potrebbe essere stabile per una coppia e divergere per un'altra. Standardizzare prima dell'SGD (poi riportare Θ alla scala originale con `theta_to_original_scale`, per poter disegnare la frontiera direttamente nelle coordinate PCA) risolve il problema senza cambiare il classificatore appreso, solo il modo in cui viene ottimizzato — lo stesso principio del ricondizionamento discusso in HW1/HW3. La loss scende da 0.693 (cioè $\log 2$, il valore atteso a $\Theta = 0$ con classi bilanciate) fino a 0.064: buona convergenza.

In [3]:

```
1 def plot_boundary(ax, theta_, z1_min, z1_max, **kw):
2     z1_r = np.linspace(z1_min, z1_max, 100)
3     z2_b = -(theta_[0] + theta_[1]*z1_r) / theta_[2]
4     ax.plot(z1_r, z2_b, **kw)
5
6 fig, ax = plt.subplots(figsize=(7,6))
7 for cls, name, color in [(0, 'cifra 3', 'tab:blue'), (1, 'cifra 4',
8     'tab:red')]:
9     mask_tr = y_train_34 == cls
10    ax.scatter(Z_train_34[mask_tr,0], Z_train_34[mask_tr,1], s=8,
11        alpha=0.4, color=color,
12        label=f'train {name}')
13 plot_boundary(ax, theta_lin_34, Z_train_34[:,0].min()-1, Z_train_34[:,0].max()+1,
14     color='k', lw=2, label='frontiera di decisione')
15 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
16 ax.set_title('Classificatore lineare sulle feature PCA (cifre 3 vs 4)')
17 ax.set_xlim(Z_train_34[:,0].min()-1, Z_train_34[:,0].max()+1)
```

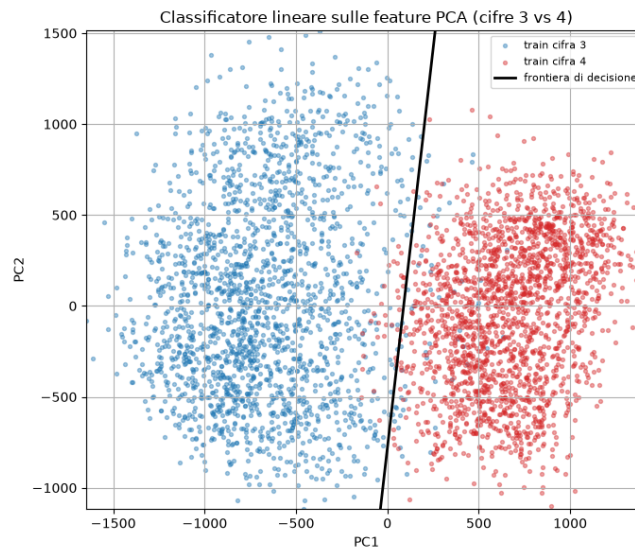


```

18 ax.set_ylim(Z_train_34[:,1].min()-1, Z_train_34[:,1].max()+1)
19 ax.legend(fontsize=8)
20 plt.tight_layout()
21 plt.show()

```

Out[3]:



In [4]:

```

1 def confusion_matrix_counts(y_true, y_pred):
2     TP = np.sum((y_true==1) & (y_pred==1))
3     FP = np.sum((y_true==0) & (y_pred==1))
4     FN = np.sum((y_true==1) & (y_pred==0))
5     TN = np.sum((y_true==0) & (y_pred==0))
6     return TP, FP, FN, TN
7
8 def eval_predictions(y_true, y_pred):
9     TP, FP, FN, TN = confusion_matrix_counts(y_true, y_pred)
10    acc = (TP+TN)/(TP+FP+FN+TN)
11    prec = TP/(TP+FP) if (TP+FP)>0 else 0.0
12    rec = TP/(TP+FN) if (TP+FN)>0 else 0.0
13    f1 = 2*prec*rec/(prec+rec) if (prec+rec)>0 else 0.0
14    return {'TP':TP, 'FP':FP, 'FN':FN, 'TN':TN, 'accuracy':acc,
15            'precision':prec, 'recall':rec, 'f1':f1}
16
17 p_test_lin_34 = sigmoid(Z_test_34_tilde @ theta_lin_34)
18 pred_lin_34 = (p_test_lin_34 >= 0.5).astype(int)
19 metrics_lin_34 = eval_predictions(y_test_34, pred_lin_34)
20 print("Classificatore lineare (3 vs 4), metriche sul test:")
21 for k, v in metrics_lin_34.items():
22     print(f"    {k}: {v:.4f}" if isinstance(v, float) else f"    {k}: {v}")

```

Out[4]:

```

1 Classificatore lineare (3 vs 4), metriche sul test:
2     TP: 491
3     FP: 10

```



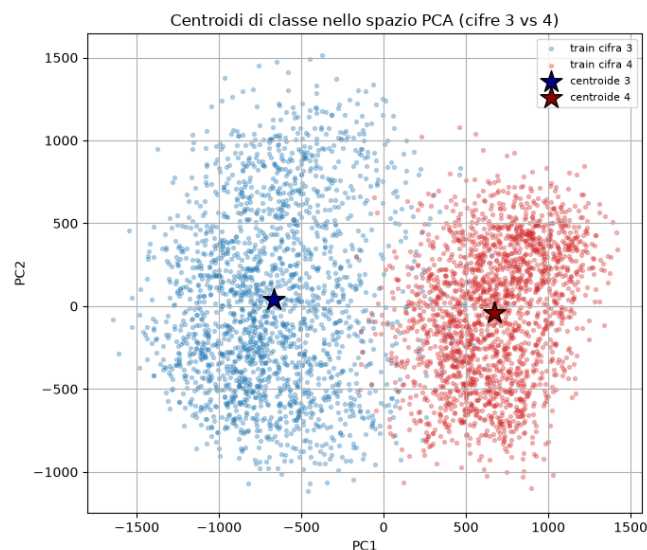
```
4 FN: 10
5 TN: 489
6 accuracy: 0.9800
7 precision: 0.9800
8 recall: 0.9800
9 f1: 0.9800
```

In [5]:

```
1 mu3_34 = Z_train_34[y_train_34 == 0].mean(axis=0)
2 mu4_34 = Z_train_34[y_train_34 == 1].mean(axis=0)
3 print("centroide cifra 3:", mu3_34)
4 print("centroide cifra 4:", mu4_34)
5
6 fig, ax = plt.subplots(figsize=(7,6))
7 for cls, name, color in [(0, 'cifra 3', 'tab:blue'), (1, 'cifra 4',
8   'tab:red')]:
9     mask_tr = y_train_34 == cls
10    ax.scatter(Z_train_34[mask_tr,0], Z_train_34[mask_tr,1], s=8,
11      alpha=0.3, color=color,
12      label=f'train {name}')
13 ax.scatter(*mu3_34, color='navy', marker='*', s=300, edgecolor='
14   black',
15   label='centroide 3', zorder=5)
16 ax.scatter(*mu4_34, color='darkred', marker='*', s=300, edgecolor='
17   black',
18   label='centroide 4', zorder=5)
19 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
20 ax.set_title('Centroidi di classe nello spazio PCA (cifre 3 vs 4)')
21 ax.legend(fontsize=8)
22 plt.tight_layout()
23 plt.show()
```

Out [5]:

```
1 centroide cifra 3: [-669.14339742  38.92997526]
2 centroide cifra 4: [669.81287556 -38.96892471]
```

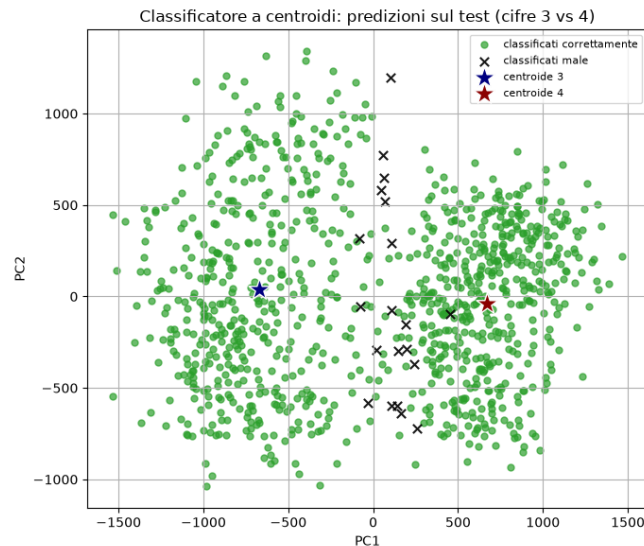


In [6]:

```
1 def centroid_classify(Z_, mu0, mu1):
2     d0 = np.linalg.norm(Z_ - mu0, axis=1)
3     d1 = np.linalg.norm(Z_ - mu1, axis=1)
4     return (d1 < d0).astype(int)    # 1 se piu' vicino a mu1
5
6 pred_centroid_34 = centroid_classify(Z_test_34, mu3_34, mu4_34)
7 metrics_centroid_34 = eval_predictions(y_test_34, pred_centroid_34)
8 print("Classificatore a centroidi (3 vs 4), metriche sul test:")
9 for k, v in metrics_centroid_34.items():
10     print(f"    {k}: {v:.4f}" if isinstance(v, float) else f"    {k}: {
11         v}")
12
13 correct_centroid = pred_centroid_34 == y_test_34
14 fig, ax = plt.subplots(figsize=(7,6))
15 ax.scatter(Z_test_34[correct_centroid,0], Z_test_34[
16     correct_centroid,1],
17     s=25, color='tab:green', label='classificati
18     correttamente', alpha=0.7)
19 ax.scatter(Z_test_34[~correct_centroid,0], Z_test_34[~
20     correct_centroid,1],
21     s=45, color='black', marker='x', label='classificati
22     male', alpha=0.9)
23 ax.scatter(*mu3_34, color='navy', marker='*', s=300, edgecolor='
24     white',
25     label='centroide 3', zorder=5)
26 ax.scatter(*mu4_34, color='darkred', marker='*', s=300, edgecolor='
27     white',
28     label='centroide 4', zorder=5)
29 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
30 ax.set_title('Classificatore a centroidi: predizioni sul test (
31     cifre 3 vs 4)')
32 ax.legend(fontsize=8)
33 plt.tight_layout()
34 plt.show()
```

Out[6]:

```
1 Classificatore a centroidi (3 vs 4), metriche sul test:
2   TP: 499
3   FP: 18
4   FN: 2
5   TN: 481
6   accuracy: 0.9800
7   precision: 0.9652
8   recall: 0.9960
9   f1: 0.9804
```



In [7]:

```
1 print(f"{'':>26} {'TP':>5} {'FP':>5} {'FN':>5} {'TN':>5} {'accuracy':>9} ")
2     f"{'precision':>10} {'recall':>8} {'f1':>7}")
3 for nome, mtr in [('lineare', metrics_lin_34), ('a centroidi',
4     metrics_centroid_34)]:
5     print(f"{nome:>26} {mtr['TP']:>5} {mtr['FP']:>5} {mtr['FN']:>5}
6         {mtr['TN']:>5} "
7         f"{mtr['accuracy']:>9.4f} {mtr['precision']:>10.4f} {mtr
8         ['recall']:>8.4f} "
9         f"{mtr['f1']:>7.4f}")
10
11 disaccordo = pred_lin_34 != pred_centroid_34
12 entrambi_sbagliano = (pred_lin_34 != y_test_34) & (pred_centroid_34
13     != y_test_34)
14 print()
15 print(f"campioni di test su cui i due metodi danno risposta diversa
16     : {np.sum(disaccordo)} "
17     f"su {len(y_test_34)}")
18 print(f"campioni sbagliati da entrambi: {np.sum(entrambi_sbagliano)}")
19 print(f"sbagliati solo dal lineare: "
20     f"{np.sum((pred_lin_34 != y_test_34) & (pred_centroid_34 ==
21     y_test_34))}")
22 print(f"sbagliati solo dai centroidi: "
23     f"{np.sum((pred_centroid_34 != y_test_34) & (pred_lin_34 ==
24     y_test_34))}")
```

Out [7]:

	TP	FP	FN	TN	accuracy	precision
	recall		f1			
lineare	491	10	10	489	0.9800	0.9800
	0.9800	0.9800				
a centroidi	499	18	2	481	0.9800	0.9652
	0.9960	0.9804				

campioni di test su cui i due metodi danno risposta diversa: 16 su 1000
campioni sbagliati da entrambi: 12

```

7 sbagliati solo dal lineare: 8
8 sbagliati solo dai centroidi: 8

```

Stessa accuracy, errori diversi — il punto centrale dello Step 4. 0.9800 per entrambi, identica al terzo decimale, ma le matrici di confusione raccontano storie diverse: il lineare è **bilanciato** (10 FP, 10 FN — sbaglia in modo simmetrico tra le due classi), i centroidi sono **sbilanciati** (18 FP, solo 2 FN — precision più bassa 0.965, recall più alta 0.996: il classificatore a centroidi tende a classificare più cose come “cifra 4”). Sui 1000 campioni di test, solo 16 vedono un disaccordo tra i due metodi, e di questi 12 sono sbagliati da *entrambi* (probabilmente ambigui anche per un occhio umano) — restano solo 8+8 = 16 campioni “di frontiera” dove i due criteri geometrici (iperpiano vs bisettrice tra centroidi) danno responso opposto.

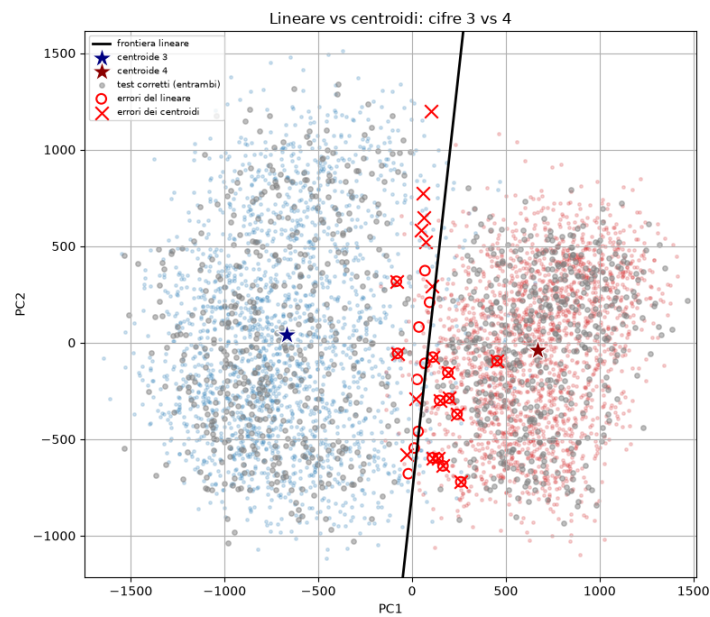
In [8]:

```

1  pred_lin_wrong = pred_lin_34 != y_test_34
2  pred_centroid_wrong = pred_centroid_34 != y_test_34
3
4  fig, ax = plt.subplots(figsize=(8,7))
5  for cls, color in [(0, 'tab:blue'), (1, 'tab:red')]:
6      mask_tr = y_train_34 == cls
7      ax.scatter(Z_train_34[mask_tr,0], Z_train_34[mask_tr,1], s=6,
8                  alpha=0.2, color=color)
9
10 plot_boundary(ax, theta_lin_34, Z_train_34[:,0].min()-1, Z_train_34[:,0].max()+1,
11               color='k', lw=2, label='frontiera lineare')
12 ax.scatter(*mu3_34, color='navy', marker='*', s=300, edgecolor='white',
13            label='centroide 3', zorder=5)
14 ax.scatter(*mu4_34, color='darkred', marker='*', s=300, edgecolor='white',
15            label='centroide 4', zorder=5)
16
17 ax.scatter(Z_test_34[~(pred_lin_wrong | pred_centroid_wrong),0],
18            Z_test_34[~(pred_lin_wrong | pred_centroid_wrong),1],
19            s=15, color='gray', alpha=0.5, label='test corretti (entrambi)')
20 ax.scatter(Z_test_34[pred_lin_wrong,0], Z_test_34[pred_lin_wrong,1], s=60, facecolors='none',
21            edgecolors='red', linewidths=1.5, label='errori del lineare')
22 ax.scatter(Z_test_34[pred_centroid_wrong,0], Z_test_34[pred_centroid_wrong,1], s=110,
23            marker='x', color='red', linewidths=1.5, label='errori dei centroidi')
24
25 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
26 ax.set_title('Lineare vs centroidi: cifre 3 vs 4')
27 # limiti fissati sui dati, altrimenti la frontiera allarga gli assi da sola
28 ax.set_xlim(Z_train_34[:,0].min()-100, Z_train_34[:,0].max()+100)
29 ax.set_ylim(Z_train_34[:,1].min()-100, Z_train_34[:,1].max()+100)
30 ax.legend(fontsize=7, loc='upper left')
31 plt.tight_layout()
32 plt.show()

```

Out [8]:



Frontiera lineare, centroidi ed errori dei due metodi sovrapposti, cifre 3 vs 4. Nota tecnica: i limiti degli assi sono fissati esplicitamente sui dati di training, altrimenti la retta di frontiera (pendenza molto grande in queste coordinate) allarga da sola gli assi e schiaccia la nuvola di punti in una striscia illeggibile.

Sia gli errori del lineare (cerchi rossi) sia quelli dei centroidi (crocette rosse) si concentrano quasi tutti nella stretta fascia centrale tra i due cluster — coerente col fatto che quasi tutti gli errori (12 su 16 in disaccordo) sono ambigui per entrambi i metodi. La frontiera lineare (nera) taglia questa fascia con un'inclinazione leggermente diversa dalla perpendicolare al segmento che unisce i due centroidi — un dettaglio geometrico che diventa più evidente nelle coppie meno simmetriche (celle successive).

In [9]:

```
1 def run_pca_classifiers(df, digit0, digit1, k=2, seed=7):
2     X_train_, y_train_, X_test_, y_test_ = load_digit_pair(df,
3         digit0, digit1, seed=seed)
4     mean_, P_, S_ = pca_fit(X_train_, k)
5     Z_train_ = pca_project(X_train_, mean_, P_)
6     Z_test_ = pca_project(X_test_, mean_, P_)
7
8     Zte_tilde = np.column_stack([np.ones(len(Z_test_)), Z_test_])
9     mu_, sigma_ = standardize_fit(Z_train_)
10    Zs_train_tilde = np.column_stack([np.ones(len(Z_train_)), (
11        Z_train_ - mu_)/sigma_])
12    theta_std_, _ = sgd_logreg(Zs_train_tilde, y_train_, np.zeros(k
13        +1),
14        eta=0.5, batch_size=32, n_epochs
15        =200, seed=seed)
16    theta_ = theta_to_original_scale(theta_std_, mu_, sigma_)
17    pred_lin_ = (sigmoid(Zte_tilde @ theta_) >= 0.5).astype(int)
18    metrics_lin_ = eval_predictions(y_test_, pred_lin_)
19
20    mu0_ = Z_train_[y_train_ == 0].mean(axis=0)
```

```

17     mu1_ = Z_train_[y_train_ == 1].mean(axis=0)
18     pred_cent_ = centroid_classify(Z_test_, mu0_, mu1_)
19     metrics_cent_ = eval_predictions(y_test_, pred_cent_)
20
21     d_, s_, ratio_ = separazione(Z_train_, y_train_)
22
23     return {'digits': (digit0, digit1), 'Z_train': Z_train_, '
24             y_train': y_train_,
25             'Z_test': Z_test_, 'y_test': y_test_, 'theta': theta_,
26             'mu0': mu0_, 'mu1': mu1_, 'centroid_dist': d_, 'spread'
27             : s_, 'sep_ratio': ratio_,
28             'var_2pc': np.sum(S_[:2]**2)/np.sum(S_**2),
29             'metrics_lin': metrics_lin_, 'metrics_cent':
30             metrics_cent_,
31             'pred_lin_wrong': pred_lin_ != y_test_, '
32             pred_cent_wrong': pred_cent_ != y_test_}
33
34 pairs = [(3,4), (1,7), (5,8), (2,3)]
35 results_pairs = {pair: run_pca_classifiers(df_mnist, *pair) for
36                  pair in pairs}
37
38 print(f"{'coppia':>8} {'dist. centr.':>13} {'d/s':>7} {'var 2PC
39       ':>9} "
40       f"{'acc lin':>8} {'prec lin':>9} {'rec lin':>8} "
41       f"{'acc cent':>9} {'prec cent':>10} {'rec cent':>9}")
42
43 for pair in pairs:
44     r = results_pairs[pair]
45     ml, mc = r['metrics_lin'], r['metrics_cent']
46     print(f"{str(pair):>8} {r['centroid_dist']:>13.1f} {r['
47           sep_ratio']:>7.3f} "
48           f"{r['var_2pc']:>9.3f} {ml['accuracy']:>8.4f} {ml['
49           precision']:>9.4f} "
50           f"{ml['recall']:>8.4f} {mc['accuracy']:>9.4f} {mc['
51           precision']:>10.4f} "
52           f"{mc['recall']:>9.4f}")

```

Out[9]:

	coppia	dist. centr.	d/s	var 2PC	acc lin	prec lin	rec lin	acc
		cent prec cent rec cent						
1	(3, 4)	1333.5	2.281	0.241	0.9810	0.9783	0.9841	
2		0.9810	0.9672	0.9960				
3	(1, 7)	1365.3	2.126	0.363	0.9830	0.9898	0.9760	
4		0.9670	0.9936	0.9399				
5	(5, 8)	550.3	0.749	0.195	0.7280	0.7399	0.6982	
6		0.7300	0.7441	0.6962				
7	(2, 3)	1077.1	1.631	0.212	0.9210	0.9173	0.9298	
8		0.9120	0.9032	0.9279				

In [10]:

```

1 fig, axes = plt.subplots(2, 2, figsize=(15, 13))
2
3 for ax, pair in zip(axes.flat, pairs):
4     r = results_pairs[pair]
5     d0, d1 = pair
6     for cls, color in [(0, 'tab:blue'), (1, 'tab:red')]:
7         mask_tr = r['y_train'] == cls

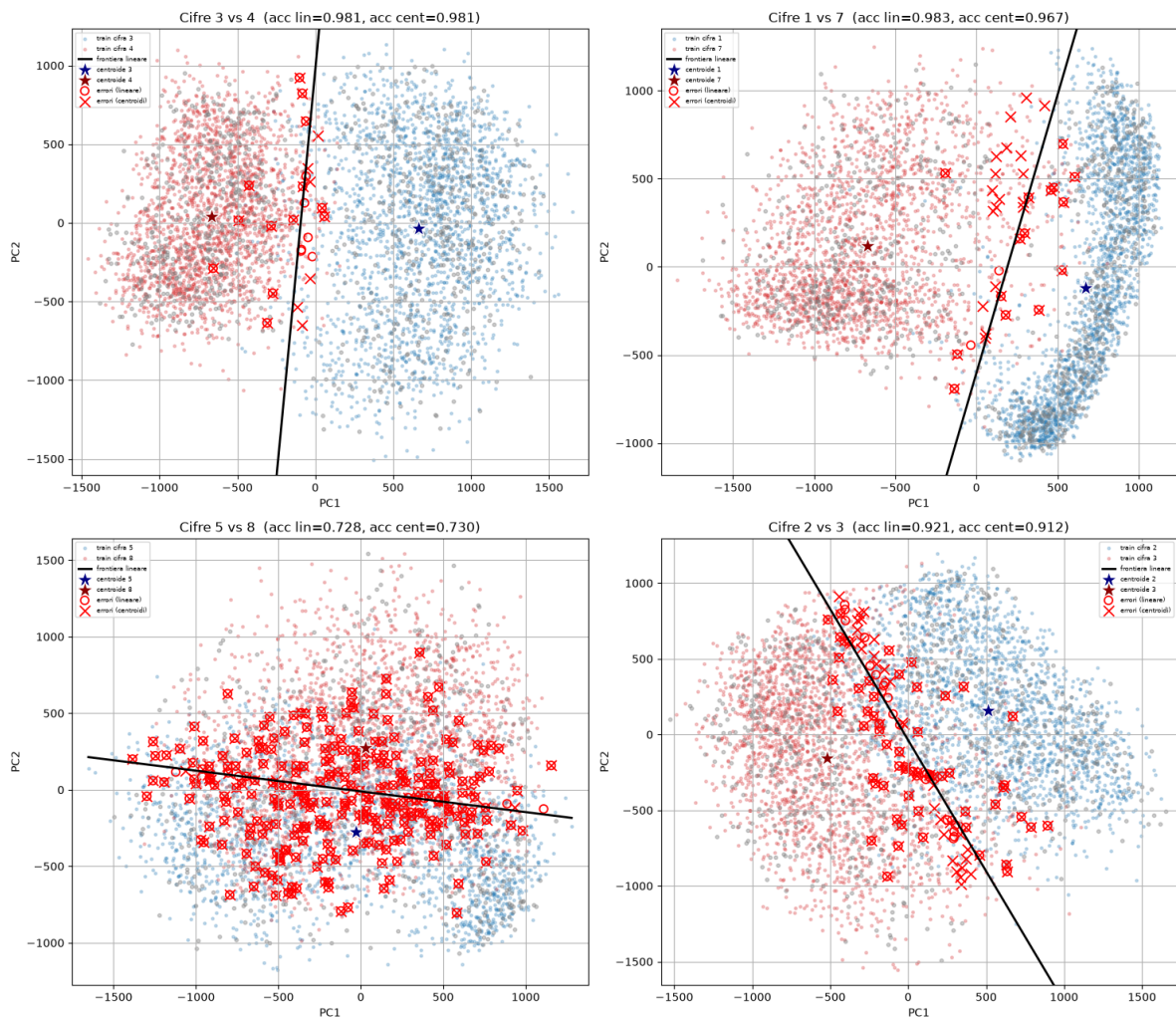
```

```

8         ax.scatter(r['Z_train'][mask_tr,0], r['Z_train'][mask_tr
9                     ,1], s=6, alpha=0.25,
10                     color=color, label=f'train cifra {pair[cls]}')
11
12     plot_boundary(ax, r['theta'], r['Z_train'][:,0].min()-1, r['
13         Z_train'][:,0].max()+1,
14                     color='k', lw=2, label='frontiera lineare')
15     ax.scatter(*r['mu0'], color='navy', marker='*', s=250,
16               edgecolor='white', zorder=5,
17               label=f'centroide {d0}')
18     ax.scatter(*r['mu1'], color='darkred', marker='*', s=250,
19               edgecolor='white', zorder=5,
20               label=f'centroide {d1}')
21
22     wrong_either = r['pred_lin_wrong'] | r['pred_cent_wrong']
23     ax.scatter(r['Z_test'][~wrong_either,0], r['Z_test'][~
24         wrong_either,1], s=10,
25               color='gray', alpha=0.4)
26     ax.scatter(r['Z_test'][r['pred_lin_wrong'],0], r['Z_test'][r['
27         pred_lin_wrong'],1], s=55,
28               facecolors='none', edgecolors='red', linewidths=1.3,
29               label='errori (lineare)')
30     ax.scatter(r['Z_test'][r['pred_cent_wrong'],0], r['Z_test'][r['
31         pred_cent_wrong'],1], s=90,
32               marker='x', color='red', linewidths=1.3, label='
33         errori (centroidi)')
34
35     ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
36     ax.set_title(f"Cifre {d0} vs {d1} (acc lin={r['metrics_lin']['
37         accuracy']:.3f}, "
38                 f"acc cent={r['metrics_cent']['accuracy']:.3f})")
39     ax.set_xlim(r['Z_train'][:,0].min()-100, r['Z_train'][:,0].max
40                 ()+100)
41     ax.set_ylim(r['Z_train'][:,1].min()-100, r['Z_train'][:,1].max
42                 ()+100)
43     ax.legend(fontsize=6, loc='best')
44
45 plt.tight_layout()
46 plt.show()

```

Out[10]:



Confronto sistematico su quattro coppie di cifre: *frontiera lineare, centroidi, errori di entrambi i metodi.*

Quale metrica ordina meglio le coppie: distanza tra centroidi o rapporto d/s ?

Guardando solo “dist. centr.”, l’ordine di difficoltà sarebbe $(5, 8) \ll (2, 3) < (3, 4) < (1, 7)$ — corretto nella direzione, ma con $(1, 7)$ e $(3, 4)$ quasi indistinguibili (1365 vs 1334) pur avendo accuratze molto diverse. Il rapporto d/s (che normalizza per la dispersione interna alle classi, introdotto nell’Esercizio 5) ordina in modo più coerente con l’accuratezza osservata: $(1, 7) = 2.126$ (il più separato in proporzione), $(3, 4) = 2.281$ — qui invertito rispetto a d da solo, ma entrambi vicini e con accuratezza quasi identica (0.983 e 0.981) — poi $(2, 3) = 1.631$ (accuracy 0.92) e infine $(5, 8) = 0.749$, l’unico caso < 1 , che crolla all’accuracy 0.73 per entrambi i metodi. La “var 2PC” invece **non** ordina bene: $(1, 7)$ ha la massima varianza spiegata (0.363) ma non è la coppia più facile, confermando quanto già discusso nell’Esercizio 5 — la varianza catturata dalla PCA non misura la separabilità tra le classi specifiche.

Confronto lineare vs centroidi su tutte e quattro le coppie. Le accuratze dei due metodi sono sorprendentemente vicine ovunque (differenza massima 0.016, per $(1, 7)$), ma con lo stesso pattern sistematico già visto per $(3, 4)$: il classificatore a centroidi ha quasi sempre precision più bassa e recall più alta (o viceversa a seconda della coppia) — $(1, 7)$: precision centroidi 0.994 molto alta ma recall solo 0.940; il lineare è più bilanciato in entrambe le direzioni.

Come la similarità tra classi influenza forma dei cluster, separazione dei centroidi, frontiere. La griglia di quattro pannelli mostra una progressione netta: (1, 7) e (3, 4) hanno cluster compatti e ben separati con pochi errori concentrati sul bordo; (2, 3) mostra già più overlap, con una fascia di errori più ampia; (5, 8) è quasi completamente mescolato, con errori (crochette e cerchi rossi) sparsi su gran parte del piano anziché concentrati su un confine netto — non esiste più una vera “frontiera” nel senso visivo del termine, solo una retta che taglia una nuvola indistinguibile.

Un dettaglio geometrico visibile solo confrontando i pannelli. La frontiera lineare (nera) **non è mai esattamente perpendicolare** al segmento che unisce i due centroidi (stellati) — lo si nota bene in (1, 7) e (2, 3), dove la retta appare visibilmente “storta” rispetto a quella congiungente. Questo è atteso: la bisettrice perpendicolare tra i centroidi è per costruzione la frontiera del classificatore a *centroidi* (dimostrabile algebricamente: il luogo $\{z : \|z - \mu_0\| = \|z - \mu_1\|\}$ è esattamente l’iperpiano $2z^T(\mu_1 - \mu_0) = \|\mu_1\|^2 - \|\mu_0\|^2$, perpendicolare a $\mu_1 - \mu_0$), mentre il classificatore **lineare** ottimizza direttamente la BCE sui dati di training, senza alcun vincolo di passare per il punto medio del segmento o di essere perpendicolare ad esso — può quindi orientarsi diversamente se la forma o la dispersione delle due classi non è simmetrica, esattamente ciò che differenzia i due metodi quando le classi non sono perfettamente “a palla” e ben bilanciate in varianza.